LLM-Based Analysis of User Comments for HarmonyOS Applications

Lin Wenqi Hu Haozhan Wang Chaohe

May 27, 2026

1 Introduction

1.1 Motivation

HarmonyOS, developed by Huawei, is a distributed operating system designed for a wide range of devices including smartphones, tablets, wearables, and IoT equipment. Since its official release, the HarmonyOS ecosystem has expanded rapidly, with the Huawei AppGallery now hosting hundreds of thousands of native HarmonyOS applications [1].

As the ecosystem matures, user comments on the AppGallery have become a critical channel for developers to understand user satisfaction, identify bugs, and prioritize feature improvements. Unlike structured bug reports or crash logs, app store reviews are unstructured, multilingual, and often mix multiple concerns within a single comment. For example, a single review might praise the UI design while simultaneously reporting a login failure on a specific device model.

Existing approaches to app review analysis largely target the Android (Google Play) and iOS (App Store) ecosystems [3, 4]. However, the HarmonyOS platform introduces unique challenges:

- **Closed ecosystem tooling.** Unlike Android's `adb`-based automation, HarmonyOS devices use the proprietary HDC (HarmonyOS Device Connector) protocol with limited tooling support.
- **Diverse device landscape.** HarmonyOS runs on a heterogeneous mix of hardware, making device-specific compatibility issues more prevalent.
- **Rapid API evolution.** The platform is evolving quickly, and compatibility problems between API versions often surface in user reviews.

These factors motivate the development of a specialized pipeline tailored to the HarmonyOS application ecosystem.

1.2 Contributions

The main contributions of this work are:

1. A **UI-automation crawler** that reliably scrapes user comments from the Huawei AppGallery on HarmonyOS devices using the `hmdriver2` library and HDC protocol.
2. An **LLM-based classification pipeline** that performs multi-dimensional sentiment analysis on user comments and extracts concrete, actionable suggestions for developers.
3. A **two-stage clustering workflow** that groups semantically similar issues within each dimension-sentiment bucket and merges cross-bucket redundancies.
4. An **interactive web dashboard** that presents aggregated results—sentiment distribution, dimension breakdown, topic clusters, and raw comments—in an intuitive visual format.
5. An **empirical dataset** covering 55 HarmonyOS applications across multiple categories, with over 27,000 scraped comments.

2 Background and Related Work

2.1 The HarmonyOS Application Ecosystem

HarmonyOS is a microkernel-based, distributed operating system first released by Huawei in 2019. Unlike Android and iOS, HarmonyOS is designed with a *device-agnostic* philosophy: a single application binary can adapt its UI and behavior across phones, tablets, foldables, smartwatches, and in-vehicle displays. The platform's native application framework, ArkUI, uses a declarative programming model with the ArkTS language (a TypeScript superset).

The Huawei AppGallery serves as the official distribution channel for HarmonyOS applications. As of 2025, the AppGallery hosts over 500,000 native HarmonyOS applications [2]. Each application page includes structured metadata (title, subtitle, category) and user-generated reviews consisting of a username, star rating (1–5), textual content, date, and device information.

2.2 Mobile App Review Analysis

Analyzing user reviews to inform software maintenance and evolution is an active research area. Early work [3] conducted an empirical study on the Apple App Store, finding that a significant portion of reviews contain actionable information for developers but are buried among uninformative ratings. A subsequent survey [4] of app review analysis techniques found that NLP-based classification of reviews by topic (bug reports, feature requests, user experience) consistently provides value to development teams.

More recently, the emergence of large language models (LLMs) has opened new possibilities for review analysis. Recent studies [5] demonstrated that GPT-4-level models can achieve high accuracy in extracting fine-grained issue types and sentiment from app reviews without task-specific training data. Our work builds on this insight by applying LLM-based analysis specifically to the HarmonyOS ecosystem.

2.3 Mobile UI Automation and Testing

Automated interaction with mobile applications is essential for both testing and data collection. On Android, tools like UIAutomator [7] and Appium [8] provide cross-platform UI automation. On HarmonyOS, the equivalent is HDC (HarmonyOS Device Connector), a command-line tool analogous to `adb`, paired with the `hmdriver2` library [6]. These tools enable programmatic UI inspection, element location via XPath, gesture simulation (tap, swipe), and screenshot capture on connected HarmonyOS devices.

In our pipeline, `hmdriver2` serves as the foundation for the comment scraper, providing the ability to navigate the AppGallery UI and extract comment data that is not accessible through any public API.

3 System Architecture

The AGCommentSpy pipeline consists of four main stages, as illustrated in Figure 1:

1. **Comment Scraping** — Automated UI navigation and text extraction from the AppGallery.
2. **LLM Classification** — Sentiment and dimension classification with actionable suggestion extraction.
3. **Topic Clustering** — Semantic grouping of classified comments into coherent issue clusters.
4. **Visualization Dashboard** — Web-based presentation of aggregated results.

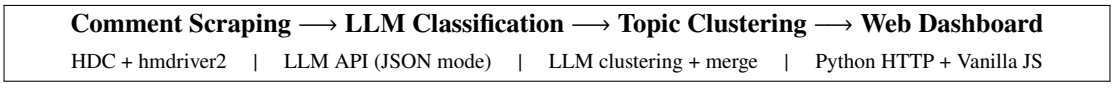

Figure 1: High-level architecture of AGCommentSpy.

3.1 Comment Scraping

The scraping module interacts with a physical HarmonyOS device via HDC. It uses the `hmdriver2` library to:

- Launch the AppGallery application and navigate to a target app's detail page using a deep-link URL of the form `https://appgallery.huawei.com/app/detail?id=<package_name>`.
- Scroll through the comment section with configurable swipe gestures (default: max 500 swipes) to load historical reviews.
- Parse each comment's UI hierarchy (dumped as JSON-then-XML) to extract the username, star rating, textual content, date, location, and device model.
- Expand truncated long comments by programmatically clicking "expand" buttons detected via XPath.
- Save raw comments as JSON files keyed by package name for downstream processing.

Figure 1 shows a simplified version of the scraping loop.

Listing 1: Simplified comment scraping loop (from `comments.py`).

```
def scrape_comments(driver, max_swipes=50, wait=2.0):
    all_comments = []
    for i in range(max_swipes):
        driver.swipe(...)           # Scroll down
        layout = driver.dump_hierarchy()
        xml = json2xml(layout)
        for node in xml.xpath(COMMENT_ITEMS):
            comment = parse_comment(node)
            all_comments.append(comment)
    return all_comments
```

3.2 LLM-Based Comment Classification

Once raw comments are collected, the classification module sends batches of comments (default: 10 per batch) to an LLM API (OpenAI-compatible). Each comment is analyzed along two dimensions:

Sentiment — Each extracted issue is labeled as `Positive`, `Negative`, or `Neutral`.

Core Dimension — Each issue is assigned to one of seven categories:

- *Functional Experience* — Core feature bugs, missing functionality, and usability of primary application flows.
- *System & Device Compatibility* — Issues specific to certain device models, screen sizes, or HarmonyOS API versions.
- *Stability & Performance* — Crashes, freezes, memory leaks, battery drain, and response lag.
- *UI & Interaction* — Visual design complaints, layout problems, navigation difficulties, and animation issues.
- *Operations & Policies* — Problems with ads, in-app purchases, account bans, content moderation, or pricing.
- *Appreciation* — General praise and positive feedback not tied to a specific feature.
- *Content Ecosystem* — Issues about content quality, availability, recommendation accuracy, or community management.

For each classified issue, the LLM also generates an **Actionable Suggestion** — a concrete recommendation that a developer could act on (e.g., "Add support for foldable screen aspect ratios in the video player"). For positive feedback, the suggestion field is set to `None`.

The system prompt instructs the LLM to output strictly valid JSON conforming to a pre-defined schema, enabling programmatic validation and error recovery. If the LLM response fails JSON parsing, an automatic repair pass is attempted before logging the batch as an error.

3.3 Topic Clustering

After per-comment classification, a two-stage clustering pipeline groups related issues:

1. **Stage 1 — Intra-bucket clustering.** Within each (dimension, sentiment) bucket, comments are sent to the LLM in batches of up to 18 items. The LLM groups items by root cause, producing clusters with a canonical issue description and a short display name.
2. **Stage 2 — Cross-bucket merging.** Clusters from Stage 1 are re-fed to the LLM to identify semantically duplicate topics across different dimension-sentiment buckets. Merged clusters receive a standardized topic name.

Both stages include automatic repair logic: if an LLM output is malformed (e.g., duplicate or missing `item_ids`), a repair pass is attempted, and as a last resort, orphaned items are placed into singleton clusters.

3.4 Web Dashboard

A lightweight web dashboard (Figure 2) provides an intuitive interface for browsing the analysis results:

- **Overview tab** — Aggregate statistics (total comments, classified count, sentiment distribution) rendered as donut and bar charts using inline SVG.
- **Comments tab** — Paginated, searchable list of raw comments with username, rating, date, location, and device metadata.
- **Analysis tab** — Dimension-filtered view showing each extracted pain point alongside its actionable suggestion and original comment context.
- **Clusters tab** — Topic clusters organized by dimension and sentiment, with expandable sample members.

The frontend is built with vanilla JavaScript (no framework dependencies) and communicates with a Python HTTP server (`server.py`) that serves static files and exposes RESTful API endpoints (`/api/apps`, `/api/apps/<pkg>/comments`, `/api/apps/<pkg>/analysis`, `/api/apps/<pkg>/clusters`).

AGCommentSpy — Comment Analysis Dashboard

Statistics	Total comments, classified count, sentiment breakdown
Charts	Donut chart (sentiment), bar chart (dimension distribution)
Comments	Paginated raw comments with full-text search
Analysis	Per-dimension pain points with LLM-generated suggestions
Clusters	Topic clusters with canonical issue descriptions

Figure 2: Structure of the web dashboard.

4 Implementation

4.1 Technology Stack

Table 1 summarizes the key technologies used in each component.

Table 1: Technology stack by component.

Component	Technology	Purpose
Device automation	`hmdriver2`, HDC	UI interaction on HarmonyOS
XML/UI parsing	`lxml`, XPath	Extract comment data from UI hierarchy
LLM client	OpenAI-compatible API	Comment classification & clustering
Batch orchestration	Python `subprocess`	Sequential multi-app scraping
Backend server	Python `http.server`	REST API for dashboard data
Frontend	Vanilla JS, SVG, CSS	Interactive visualization
Data storage	JSON files (per-app directories)	Raw, classified, and clustered data
Language	Python 3.13	All backend logic
Target platform	HarmonyOS (OpenHarmony 4/5)	Device under automation

4.2 LLM Configuration

The LLM client (`llm_client.py`) communicates with any OpenAI-compatible API endpoint. Key design decisions include:

- **JSON mode.** The API is called with `response_format: {type: "json_object"}` to enforce structured output. If the endpoint does not support this parameter, the call is retried without it.
- **Batch processing.** Comments are sent in batches (configurable batch size, default 10) to limit prompt length and improve classification quality. Each batch is processed independently.
- **Error recovery.** A three-tier recovery strategy handles malformed outputs: (1) JSON extraction from markdown code fences, (2) heuristic brace-based extraction, and (3) a secondary LLM repair call with a dedicated repair prompt.
- **Idempotent output.** Each classified item retains the original comment ID (e.g., `com.tencent.wechat_42`), enabling traceability from dashboard back to raw comments.

4.3 Deduplication and Normalization

During clustering, identical issues from different source comments are deduplicated by normalizing text (lowercasing, whitespace collapsing, Unicode normalization) and comparing the (dimension, sentiment, normalized pain point) triple. This prevents semantically identical issues from being counted multiple times while preserving traceability to individual source comments.

4.4 Project Structure

Listing 2: Project directory layout.

```
project/
  app.json                  # 55 target app definitions
  config.py                 # Device ID, screen bounds, XPath constants
  main.py                   # Single-app scraper entry point
  comments.py               # Comment parsing & scrolling logic
  analyze_comments.py       # LLM classification pipeline
  llm_client.py             # OpenAI-compatible API client
  cluster_and_label.py      # Item collection & text normalization
  server.py                 # HTTP server with REST API
  run_batch.py              # Batch orchestrator
  patch_dates.py            # Date normalization utility
  layout_output.py          # Layout XML/JSON dump helpers
  xml_utils.py              # JSON-to-XML conversion & XMLElement wrapper
```

```
  frontend/
    index.html              # Dashboard entry point
    css/style.css           # Dashboard styles
    js/api.js               # API client
    js/charts.js             # SVG chart rendering
    js/app.js               # Dashboard application logic
  scripts/
    aggregate_classified.py  # Aggregate raw LLM responses
    llm_cluster_and_label.py # Stage-1 clustering
    refine_and_merge_topics.py # Stage-2 cross-bucket merging
  com.<package>/             # Per-app output directories
    app_info.json
    comments.json
    classified_comments.json
    classified_comments.raw.json
    clusters_preview_llm.json
```

5 Dataset

The scraping pipeline was executed against 55 HarmonyOS applications listed in Table 2. The selection covers a diverse range of categories:

- **Social & Communication:** WeChat, QQ, Douyin (TikTok), Kuaishou, Xiaohongshu (RED), Weibo, DingTalk, Tencent Meeting
- **E-commerce:** Taobao, JD.com, Pinduoduo, Xianyu (Idle Fish), Vipshop, Dewu
- **Finance:** Alipay, ABC (Agricultural Bank of China), CCB, ICBC, Bank of China
- **Entertainment & Media:** Bilibili, iQiyi, Youku, Tencent Video, QQ Music, Kugou Music, Ximalaya, Tomato Novel, Red Fruit Short Drama
- **Tools & Services:** Amap (Gaode), Baidu Map, Baidu Search, Baidu Netdisk, Quark Browser, UC Browser, QQ Browser, WPS Office
- **Lifestyle & Travel:** Meituan, Dianping, Ctrip, Qunar, DiDi, 12306 Railway
- **Games:** Honor of Kings, Game for Peace, Teamfight Tactics, Subway Surfers, Happy Match
- **AI & Creativity:** Doubao (ByteDance AI assistant), Jianying (CapCut)

Table 2: Partial list of scraped applications (*full list in* `app.json`).

App Name	Package Name	Category
WeChat	`com.tencent.wechat`	Social
Douyin	`com.ss.hm.ugc.aweme`	Social
Alipay	`com.alipay.mobile.client`	Finance
Taobao	`com.taobao.taobao4hmos`	E-commerce
Meituan	`com.sankuai.hmeituan`	Lifestyle
Bilibili	`yylx.danmaku.bili`	Entertainment
Amap	`com.amap.hmapp`	Navigation
Honor of Kings	`com.tencent.tmgp.sgame.hw`	Gaming

For each application, up to 500 comment swipes were performed. The total raw dataset comprises over 27,000 individual comments, though the actual count per app varies depending on the volume of reviews available on the AppGallery.

6 Evaluation and Discussion

6.1 Classification Quality

The LLM classifier assigns each extracted issue to one of seven dimensions and three sentiment categories. The seven-dimension taxonomy was designed to balance granularity with actionability:

- Dimensions like *Stability & Performance* and *System & Device Compatibility* target issues that require engineering investigation.
- *UI & Interaction* and *Functional Experience* capture user-facing quality concerns.
- *Operations & Policies* and *Content Ecosystem* cover business and content-side concerns.
- *Appreciation* serves as a catch-all for positive feedback that does not require developer action.

By requiring the LLM to output `Actionable_Suggestion` for every negative entry, the pipeline ensures that each finding is paired with a concrete improvement proposal. Positive and neutral entries do not generate suggestions, preventing noise in the developer-facing output.

6.2 Clustering Effectiveness

The two-stage clustering approach addresses a key limitation of flat classification: multiple comments about the same underlying issue (e.g., "app crashes when opening on Mate 60 Pro") would otherwise appear as scattered entries across the dashboard. By grouping them into root-cause clusters, the dashboard presents a more actionable view of the most impactful issues.

The automatic repair and deduplication mechanisms proved essential in practice, as the LLM occasionally produced outputs with duplicate or missing item assignments. The auto-repair pipeline ensures robustness: malformed clusters are repaired automatically, and any items that cannot be assigned are surfaced as singletons rather than silently dropped.

6.3 Limitations

Several limitations should be acknowledged:

1. **Scraping reliability.** UI automation is inherently fragile. Changes to the AppGallery UI layout (XPath selectors) can break the scraper and require manual selector updates in `config.py`.
2. **LLM dependence.** The classification and clustering quality depends on the underlying LLM. Different models or API providers may produce varying quality, and prompt engineering is model-specific.
3. **Single-language focus.** The current system prompt and taxonomy are designed for Chinese-language comments. Extending to multilingual analysis would require prompt adaptation.
4. **No ground-truth evaluation.** As a course project, we did not conduct a formal human-annotated evaluation of classification accuracy. Future work should include a manually labeled subset for quantitative evaluation.
5. **Device requirement.** The scraper requires a physical HarmonyOS device connected via HDC, limiting scalability compared to API-based collection methods.

7 Conclusion and Future Work

This paper presented AGCommentSpy, a complete pipeline for collecting and analyzing user comments from the Huawei AppGallery on HarmonyOS. The system demonstrates that combining UI automation with LLM-based natural language understanding can effectively transform unstructured app store reviews into actionable developer insights. The web dashboard provides an accessible interface for browsing sentiment distributions, dimension breakdowns, and topic clusters across 55 real-world applications.

Several directions for future work are apparent:

1. **Automated labeling for evaluation.** Build a human-annotated gold standard subset to quantitatively measure classification precision and recall.

2. **Multilingual support.** Extend the taxonomy and prompts to support English and other languages present in international AppGallery listings.
3. **LLM comparison.** Benchmark multiple LLM providers and model sizes on the same dataset to identify cost-quality trade-offs.
4. **Incremental scraping.** Support scraping only new comments since the last run, reducing processing overhead for ongoing monitoring.
5. **Automated issue reporting.** Integrate with developer workflow tools (e.g., GitHub Issues, Jira) to automatically file actionable findings from the analysis pipeline.

Acknowledgments

This project builds on the open-source `hmdriver2` library [6] for HarmonyOS device interaction. We thank the contributors of that project for enabling programmatic access to HarmonyOS devices.

References

[1] Huawei Technologies Co., Ltd. *HarmonyOS Developer Guide*. https://developer.huawei.com/consumer/en/harmonyos/, 2025.

[2] Huawei Technologies Co., Ltd. *AppGallery Connect*. https://developer.huawei.com/consumer/en/appgallery/, 2025.

[3] D. Pagano and W. Maalej. User feedback in the appstore: An empirical study. In *Proc. 21st IEEE International Requirements Engineering Conference (RE)*, pp. 125–134, 2013.

[4] W. Martin, F. Sarro, Y. Jia, Y. Zhang, and M. Harman. A survey of app store analysis for software engineering. *IEEE Transactions on Software Engineering*, 43(9):817–847, 2017.

[5] Z. Zou, Y. Zhao, Y. Liu, et al. Large language models for app review analysis: An exploratory study. In *Proc. ACM International Conference on the Foundations of Software Engineering (FSE)*, 2024.

[6] CodeMatrix. `hmdriver2`: Python client for HarmonyOS HDC. https://github.com/codematrixer/hmdriver2, 2025.

[7] Google LLC. *UI Automator — Android Developers*. https://developer.android.com/training/testing/other-components/ui-automator, 2025.

[8] Appium Contributors. *Appium: Automation for Apps*. https://appium.io/, 2025.